VECTOR SEARCH BENCHMARK REVIEW

EDB Vector Plus, pgvector, VectorChord and ClickHouse on 1M and 2M rows of 768-dimension embeddings.

Postgres 18.6 · WarehousePG 7.6 · ClickHouse 26.1 · 22 to 23 October 2026

Source: six vp-bench-1 exports (runs 1 to 6) from the Vector Plus demo dashboard

EDB POSTGRES AI

©EDB 2026 — ALL RIGHTS RESERVED.

IVFPLUS IS FASTER AND HALF THE SIZE, BUT TOPS OUT AT 0.886 ON POSTGRES

3.1×
faster than IVFFlat on WarehousePG at 90% recall: 5.6 ms against 17.5 ms

5.7×
faster than IVFFlat on Postgres at 85% recall: 6.9 ms against 39.4 ms

2.1×
smaller index than IVFFlat at 2M rows: 3.55 GiB against 7.64 GiB

Still open

ivfplus on Postgres plateaus at 0.886 recall. Probes 64 to 256 add latency, up to 51 ms, and no recall.

ClickHouse and ivfplus on WarehousePG were not extended. Their ladders end at 0.76 (ef_search 128) and 0.936 (32 probes).

Runs 3 and 4 carry latency spikes. The 2M figures keep the quietest of four runs.

©EDB 2026 — ALL RIGHTS RESERVED.

THE TESTS

Engines, data, queries and index builds

©EDB 2026 — ALL RIGHTS RESERVED.

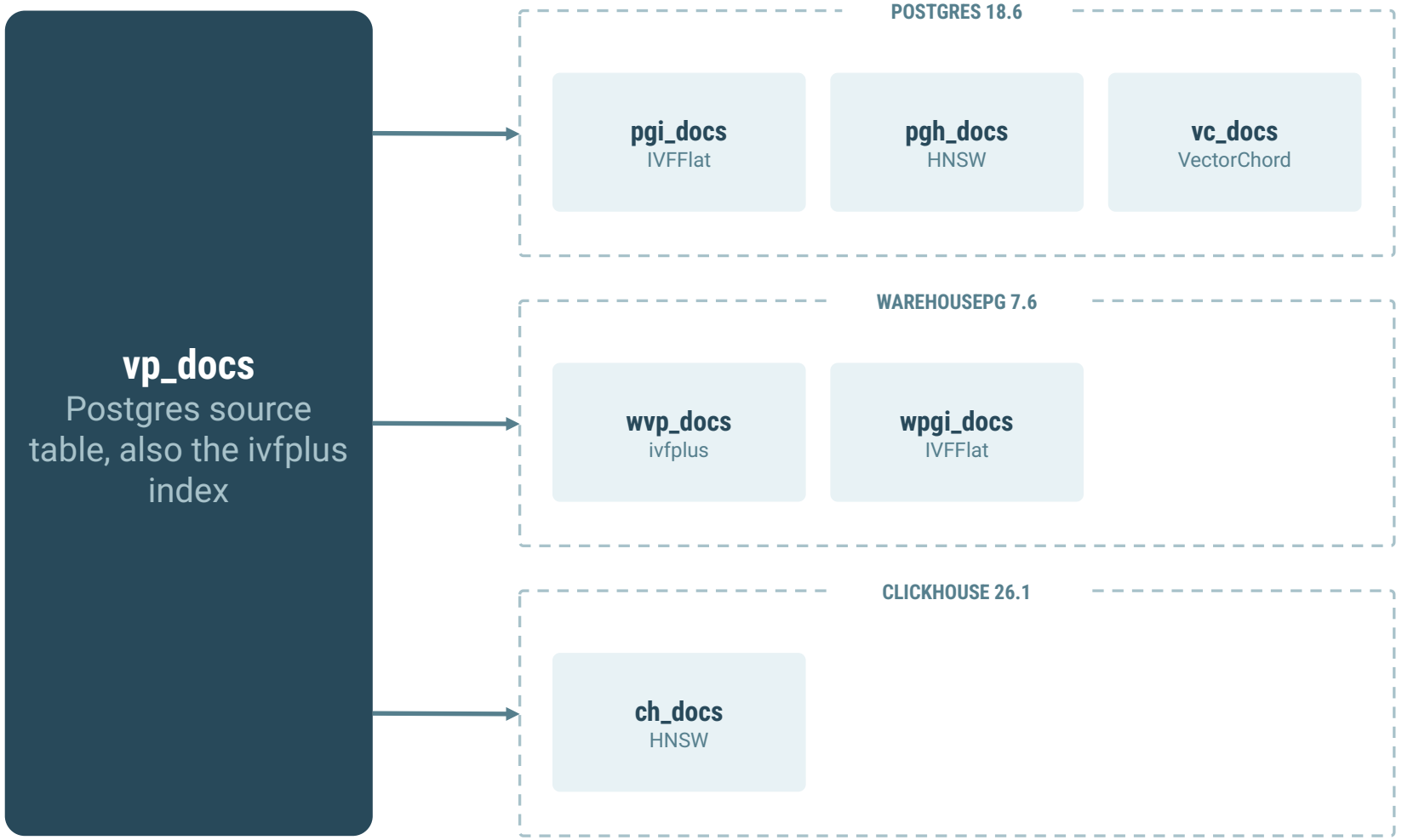
EACH ENGINE RAN ITS OWN INDEX ON ITS OWN COPY OF THE DATA

ENGINE	WHERE IT RAN	INDEX	BUILD SETTINGS
EDB Vector Plus	Postgres 18.6, edb_vectorplus 0.0.5	ivfplus (IVF with RaBitQ)	lists 1414, rotation on
EDB ivfplus @ WHPG	WarehousePG 7.6, 2 segments, optimizer off	ivfplus	lists 1414
pgvector IVFFlat	Postgres 18.6, pgvector 0.8.6	ivfflat	lists 1414
pgvector IVFFlat @ WHPG	WarehousePG 7.6, pgvector 0.7.4	ivfflat	lists 1414
pgvector HNSW	Postgres 18.6, pgvector 0.8.6 (1M only)	hnsw	m 16, ef_construction 64
VectorChord	Postgres 18.6, vchord 1.1.1	vchordrq	lists [1414]
ClickHouse	26.1.2.1 EDB PoC preview build (2M only)	vector_similarity (HNSW, bf16)	M 16, ef_construction 128

Lists are 1000 at 1M rows and 1414 at 2M, the square root of the row count. Postgres builds ran with maintenance_work_mem 12 GB, no parallel workers and shared_buffers 128 MB.

©EDB 2026 — ALL RIGHTS RESERVED.

EVERY ENGINE HOLDS THE SAME 2,000,000 ROWS



The rows

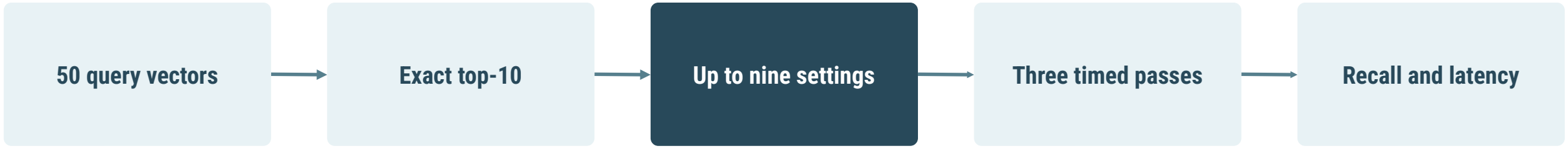
Synthetic documents on ten topics. Each embedding is the normalized mean of random word vectors plus noise, 768 dimensions.

The check

Before each comparison the app counts rows and compares the id range and an id checksum on every table. A mismatch stops the run.

©EDB 2026 — ALL RIGHTS RESERVED.

EVERY SWEEP IS SCORED AGAINST AN EXACT SCAN OF THE SAME 50 QUERIES



Stored rows picked with a fixed seed, plus Gaussian noise of 0.05, then renormalized.

A full sequential scan with index scans off. The ground truth for recall, computed once and reused.

Probes 1, 2, 4 and so on to 32, extended to 256 in the last sweep. HNSW and ClickHouse get ef_search = max(k, 4 × probes).

One untimed warm-up pass first. Engines are interleaved, indexes prewarmed, JIT off.

Recall@10 is the share of the exact top-10 returned. Latency is each query's median over the passes.

Each result is the mean over the 50 queries. Recall carries a standard error of 0.01 to 0.05, and p95 is the 95th percentile across queries.

©EDB 2026 — ALL RIGHTS RESERVED.

ONE QUERY SHAPE RUNS ON EVERY ENGINE

Postgres and WarehousePG

```
SET ivfplus.probes = 16;
-- ivfflat.probes = 16 | vchordrq.probes = '16'
-- hnsw.ef_search = 64

SELECT id, topic, category_id, title,
       embedding <-> '[0.0412, -0.0157, …]' AS distance
FROM public.vp_docs
ORDER BY embedding <-> '[0.0412, -0.0157, …]'
LIMIT 10;
```

Approximate runs set enable_seqscan = off and jit = off, so the planner cannot quietly scan the table.
The exact baseline runs the same SELECT with enable_indexscan and enable_bitmapscan off.

ClickHouse

```
SELECT id, topic, category_id, title,
       L2Distance(embedding, [0.0412, …])
         AS distance
FROM default.ch_docs
ORDER BY L2Distance(embedding, [0.0412, …])
LIMIT 10
SETTINGS hnsw_candidate_list_size_for_search = 64;
```

The vector is a literal of 768 numbers. Before each run the app checks the plan and records which index was used.

©EDB 2026 — ALL RIGHTS RESERVED.

ONE LIST OF PROBES SETS THE EFFORT FOR EVERY ENGINE

ENGINE	SETTING	VALUE FOR PROBES P	MEANING
EDB Vector Plus	ivfplus.probes	p, capped at lists	lists scanned per query
pgvector IVFFlat	ivfflat.probes	p, capped at lists	lists scanned per query
VectorChord	vchordrq.probes	'p', capped at lists	lists scanned per query
pgvector HNSW	hnsw.ef_search	max(k, 4 × p)	candidate list size
ClickHouse	hnsw_candidate_list_size_for_search	max(k, 4 × p)	candidate list size
WarehousePG twins	same as their Postgres engine	p, on each of 2 segments	more total work per query

Runs 3 to 5 used p = 1 to 32, which gives HNSW and ClickHouse ef_search 10, 16, 32, 64, 128. Run 6 extended the ladder to 256 probes, where ef_search reaches its cap of 1000. The 1M sweep predates the shared list and used each engine's own list.

©EDB 2026 — ALL RIGHTS RESERVED.

EACH INDEX WAS BUILT ONCE, WITH LISTS SET TO √ROWS

```
CREATE INDEX … USING ivfplus   (embedding vector_l2_ops) WITH (lists = 1414, rotation = on);
CREATE INDEX … USING ivfflat   (embedding vector_l2_ops) WITH (lists = 1414);
CREATE INDEX … USING hnsw      (embedding vector_l2_ops) WITH (m = 16, ef_construction = 64);
CREATE INDEX … USING vchordrq  (embedding vector_l2_ops)
               WITH (options = $$ [build.internal]  lists = [1414] $$);

ALTER TABLE ch_docs ADD INDEX vi embedding
    TYPE vector_similarity('hnsw', 'L2Distance', 768, 'bf16', 16, 128)
    GRANULARITY 100000000;
ALTER TABLE ch_docs MATERIALIZE INDEX vi;
```

Build time is one CREATE INDEX, or for ClickHouse ADD INDEX plus MATERIALIZE INDEX, measured once per engine.
Postgres built serially (no parallel workers). WarehousePG builds on its 2 segments at the same time.

©EDB 2026 — ALL RIGHTS RESERVED.

THE RESULTS

1M and 2M rows, recall against latency

©EDB 2026 — ALL RIGHTS RESERVED.

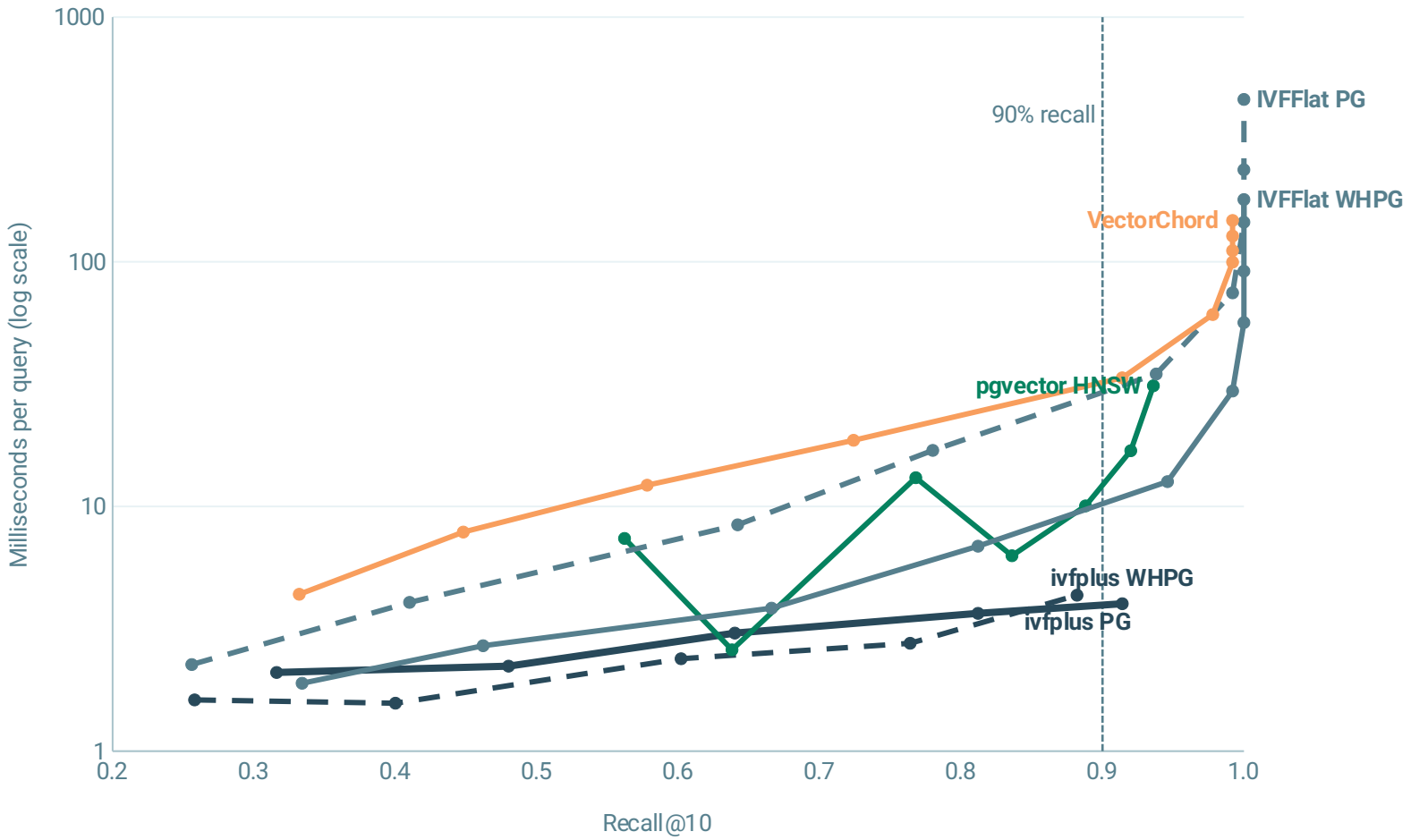
SIX SWEEPS; THE 2M FIGURES USE THE QUIETEST OF FOUR RUNS

RUN	SAVED	ROWS	ENGINES	WHAT TO KNOW
1	22 Oct, 11:12	1M	6	Includes HNSW; each engine used its own setting list; a single run.
2	22 Oct, 16:09	2M	5	Set aside: the WarehousePG IVFFlat index was rebuilt afterwards (recall at 16 probes 0.928, then 0.916).
3	23 Oct, 06:33	2M	6	First sweep with ClickHouse; the exact baseline was computed here.
4	23 Oct, 07:54	2M	6	ClickHouse again; the baseline was reused.
5	23 Oct, 08:19	2M	5	No ClickHouse; the baseline was reused; the cleanest latency of runs 3 to 5.
6	23 Oct, 10:28	2M	4	Ladder extended to 256 probes; no ClickHouse or WarehousePG ivfplus; reproduces run 5.

Recall is identical in runs 3 to 6, because the queries, the exact baseline and the indexes did not change. For each 2M setting the figure used here is the lowest latency of the runs that measured it; settings above 32 probes come from run 6 alone.

©EDB 2026 — ALL RIGHTS RESERVED.

AT 1M ROWS IVFPLUS ON WAREHOUSEPG REACHED 90% RECALL IN 4 MS



4.0 ms

ivfplus on WarehousePG at 91% recall, 16 probes

12.6 ms

IVFFlat on WarehousePG at 95% recall

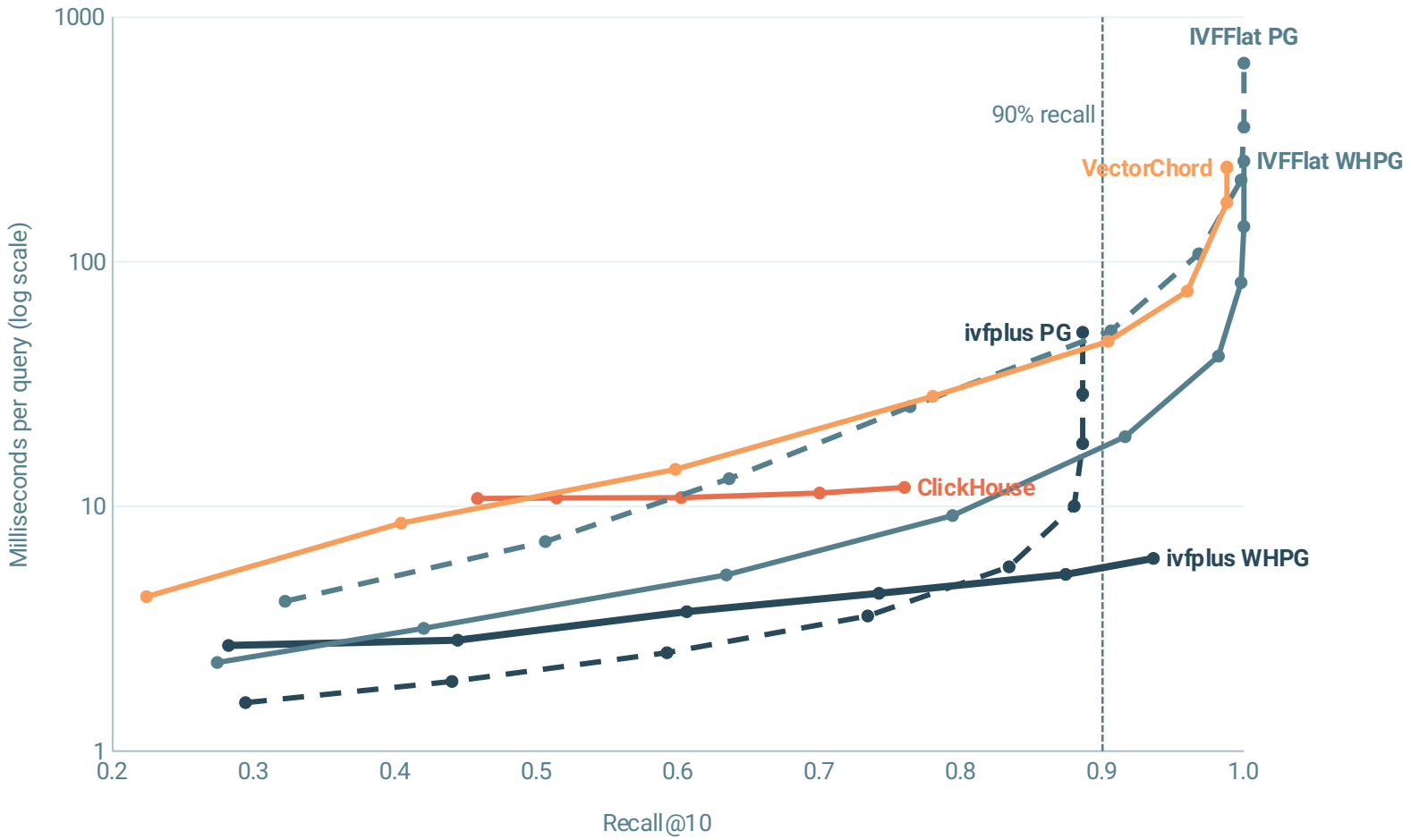
16.9 ms

pgvector HNSW at 92% recall, ef_search 320

ivfplus on Postgres reached 0.88 after 16 probes, the end of its list. At 2M it plateaus at 0.886.

©EDB 2026 — ALL RIGHTS RESERVED.

AT 2M ROWS THE ORDER HOLDS: WAREHOUSEPG IVFPLUS STAYS UNDER 7 MS



6.1 ms

ivfplus on WarehousePG at 94% recall, 32 probes

19.3 ms

IVFFlat on WarehousePG at 92% recall

47.3 ms

VectorChord at 90% recall; IVFFlat on Postgres took 52.2 ms

ivfplus on Postgres plateaus at 0.886 and ClickHouse stopped at 0.76; neither reached 90%.

©EDB 2026 — ALL RIGHTS RESERVED.

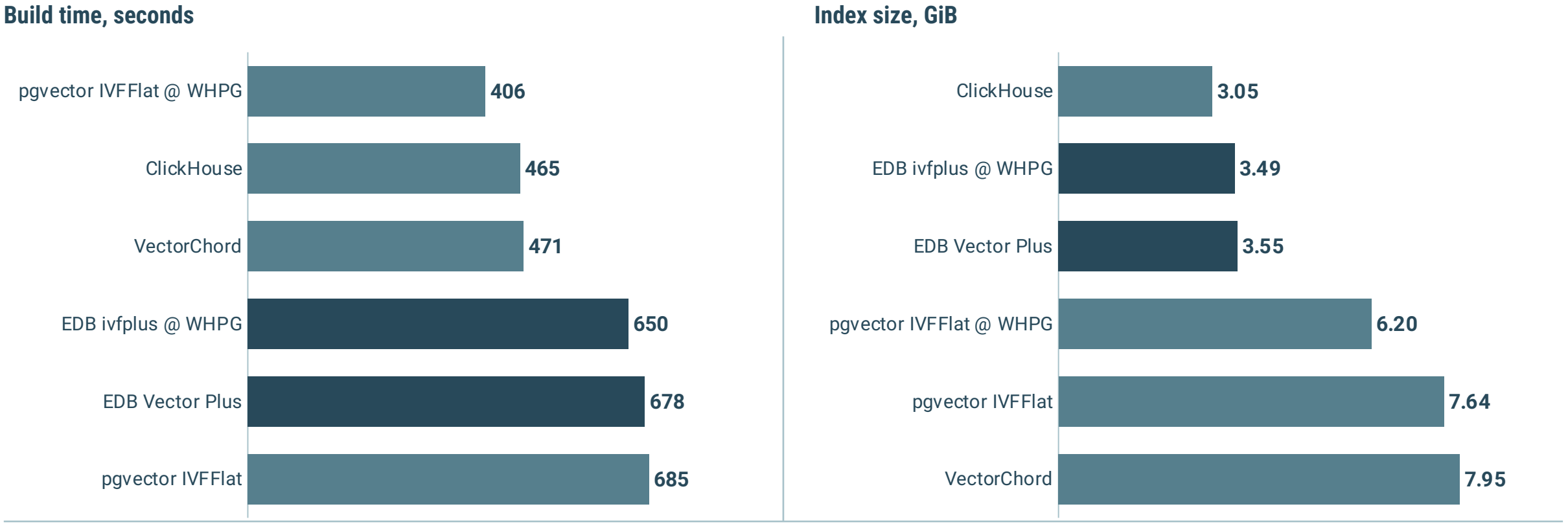
AT EQUAL RECALL IVFPLUS IS 1.3 TO 6.2X FASTER THAN IVFFLAT

LATENCY AT EQUAL RECALL, 2M ROWS	60%	75%	85%	90%	95%	99%
EDB ivfplus @ WHPG	3.7 ms	4.5 ms	5.1 ms	5.6 ms	not reached	not reached
pgvector IVFFlat @ WHPG	4.8 ms	7.9 ms	12.9 ms	17.5 ms	28.5 ms	58.2 ms
EDB Vector Plus	2.6 ms	3.8 ms	6.9 ms	not reached	not reached	not reached
pgvector IVFFlat	11.0 ms	23.8 ms	39.4 ms	50.7 ms	87.3 ms	179 ms
VectorChord	14.3 ms	25.2 ms	37.8 ms	46.5 ms	69.7 ms	not reached
ClickHouse	10.9 ms	11.9 ms	not reached	not reached	not reached	not reached
ivfplus vs IVFFlat on WarehousePG	**1.3×**	**1.8×**	**2.5×**	**3.1×**	–	–
ivfplus vs IVFFlat on Postgres	**4.3×**	**6.2×**	**5.7×**	–	–	–

Interpolated along each measured curve, lowest of the repeats. ivfplus on Postgres plateaus at 0.886, so its comparison ends there; “not reached” means the engine never got that far.

©EDB 2026 — ALL RIGHTS RESERVED.

IVFPLUS INDEXES ARE HALF THE SIZE; BUILD TIME MATCHES IVFFLAT



Per row: ivfplus 1,908 bytes, IVFFlat 4,100, VectorChord 4,268, ClickHouse 1,637. A raw 768-dimension vector is 3,072 bytes. Builds are single measurements; the WarehousePG builds ran on 2 segments.

©EDB 2026 — ALL RIGHTS RESERVED.

DOUBLING THE ROWS ADDS 32 TO 56% LATENCY AND 2.5 TO 3.1X BUILD TIME

ENGINE	LATENCY AT 16 PROBES	RECALL AT 16 PROBES	BUILD TIME	INDEX SIZE
EDB Vector Plus	4.3 → 5.7 ms (×1.31)	0.882 → 0.834	×2.5	×2.0
EDB ivfplus @ WHPG	4.0 → 5.3 ms (×1.32)	0.914 → 0.874	×3.1	×1.9
pgvector IVFFlat	34.8 → 52.2 ms (×1.50)	0.938 → 0.906	×2.5	×2.0
pgvector IVFFlat @ WHPG	12.6 → 19.3 ms (×1.53)	0.946 → 0.916	×3.0	×2.0
VectorChord	33.6 → 47.3 ms (×1.41)	0.914 → 0.904	×3.1	×2.0

Doubling the rows doubles the index and multiplies build time by 2.5 to 3.1. Query latency grows by less than the data, but recall at the same probes falls by 0.01 to 0.05, so a larger table needs a few more probes for the same recall. The 1M figures are a single run.

©EDB 2026 — ALL RIGHTS RESERVED.

WAREHOUSEPG CUTS IVFFLAT LATENCY 2.5X, BUT IVFPLUS MAINLY GAINS RECALL

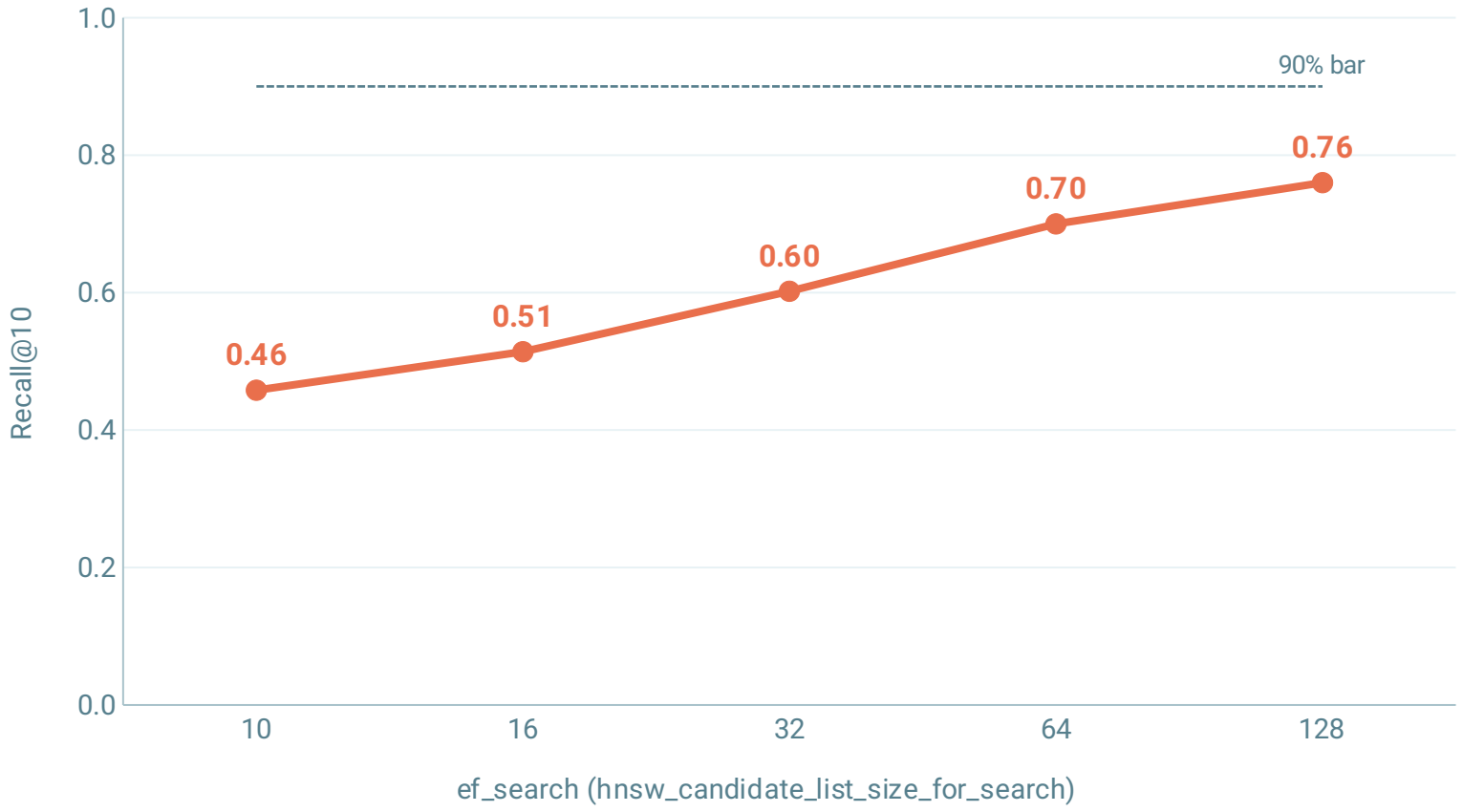
ENGINE AND PROBES	POSTGRES	WAREHOUSEPG	LATENCY GAIN	RECALL GAIN
ivfplus, 16 probes	5.7 ms · recall 0.834	5.3 ms · recall 0.874	1.07×	+0.040
ivfplus, 32 probes	10.0 ms · recall 0.880	6.1 ms · recall 0.936	1.64×	+0.056
IVFFlat, 16 probes	52.2 ms · recall 0.906	19.3 ms · recall 0.916	2.71×	+0.010
IVFFlat, 32 probes	107.7 ms · recall 0.968	41.1 ms · recall 0.982	2.62×	+0.014

IVFFlat spends 50 ms or more per query on Postgres, so splitting its lists over two segments pays off. ivfplus already answers in 5 to 10 ms there, which leaves little for a second segment to save.

Probes apply on each segment, so the same setting searches more lists in total and returns higher recall. Compare at equal recall. WarehousePG also runs Postgres 12 with pgvector 0.7.4.

©EDB 2026 — ALL RIGHTS RESERVED.

CLICKHOUSE HAS NOT REACHED 90% RECALL; ITS LATENCY IS FLAT AT 11 MS



Two latency spikes (68 and 76 ms) hit one setting in each run and did not repeat.

0.76

best recall, at ef_search 128. pgvector HNSW at 1M rows needed ef_search 640 to reach 0.94.

11 ms

at every setting from 10 to 128. A fixed per-query cost, the HTTP round trip included, seems to dominate.

465 s

to build, with a 3.05 GiB index (M 16, ef_construction 128, bf16).

©EDB 2026 — ALL RIGHTS RESERVED.

AT A 90% BAR WAREHOUSEPG IVFPLUS RANKS FIRST; AT 95% IVFFLAT TAKES OVER

RANK	ENGINE	SCORE	LATENCY AT BAR	P95	BEST RECALL	BUILD	INDEX
1	**EDB ivfplus @ WHPG**	**90.9**	**6.1 ms**	**10.9 ms**	0.936	650 s	3.49 GiB
2	**pgvector IVFFlat @ WHPG**	**53.3**	19.3 ms	38.5 ms	1.000	406 s	6.20 GiB
3	**VectorChord**	**38.4**	47.3 ms	133 ms	0.988	471 s	7.95 GiB
4	**pgvector IVFFlat**	**34.3**	52.2 ms	83.3 ms	1.000	685 s	7.64 GiB
DNQ	ClickHouse	(39.7)	not reached		0.760	465 s	3.05 GiB
DNQ	EDB Vector Plus	(34.1)	not reached		0.886	678 s	3.55 GiB

Score: latency 40, p95 10, best recall 10, build 15, index size 15; maintenance was not measured. An engine that never reaches the bar is DNQ and ranks below every engine that does.
IVFFlat on WarehousePG leads at a 95% bar (91.5) and at a 99% bar (91.5). ivfplus on WarehousePG (best 0.936) is DNQ there because its ladder ended at 32 probes; at 99% only the two IVFFlat engines qualify.

©EDB 2026 — ALL RIGHTS RESERVED.

RUNS 3 AND 4 CARRY LATENCY SPIKES; RUN 6 REPRODUCES RUN 5

SAME SETTING, FOUR RUNS	RUN 3	RUN 4	RUN 5	RUN 6
EDB Vector Plus, 16 probes	**66.5 ms**	5.9 ms	5.8 ms	5.7 ms
VectorChord, 8 probes	**427 ms**	**186 ms**	28.3 ms	28.2 ms
pgvector IVFFlat, 32 probes	**231 ms**	110 ms	108 ms	108 ms
IVFFlat @ WHPG, 32 probes	43.2 ms	**224 ms**	45.3 ms	41.1 ms

Run 6 repeats run 5's settings from 1 to 32 probes: 22 of 24 latencies agree within 10%, the median ratio is 0.99, and recall is identical. Run 6 carries no warning that ClickHouse was running, and run 5 had no ClickHouse engine.
In runs 3 and 4, 7 and 6 of 30 settings ran more than 1.5 times slower than in run 5; both included ClickHouse on the same host. This deck keeps the lowest latency per setting.

©EDB 2026 — ALL RIGHTS RESERVED.

WHAT IS STILL OPEN

Gaps in the data and the next sweep

©EDB 2026 — ALL RIGHTS RESERVED.

TWO LADDERS STILL END EARLY: IVFPLUS ON WAREHOUSEPG AND CLICKHOUSE

NOT MEASURED YET

ivfplus on WarehousePG past 32 probes: recall was still rising at 0.936.

ClickHouse above ef_search 128: recall was still rising at 0.76.

Why ivfplus on Postgres stops at 0.886: the cause is not yet known.

pgvector HNSW at 2M rows, and churn: inserts, updates and deletes.

LIMITS OF WHAT WAS MEASURED

Settings above 32 probes: one run each, so no repeat to take the lowest of.

Synthetic embeddings, 50 queries: recall standard errors of 0.01 to 0.05.

ClickHouse shared the host with Postgres in runs 3 and 4.

WarehousePG runs Postgres 12: with pgvector 0.7.4, an older stack.

©EDB 2026 — ALL RIGHTS RESERVED.

THE NEXT SWEEP SHOULD FINISH TWO LADDERS AND EXPLAIN ONE CEILING

01

Finish the ladder

Probes 64 to 256 for ivfplus on WarehousePG, and ef_search up to 1000 for ClickHouse.

02

Explain the ceiling

Rebuild ivfplus on Postgres with other lists, rotation and store_vectors settings, and see whether recall passes 0.9.

03

Repeat on a quiet host

Run each sweep three times with ClickHouse stopped, using the Repeats and quiet-host options now in the dashboard.

04

Fill the gaps

pgvector HNSW at 2M rows, a churn run for maintenance cost, then 10M rows.

©EDB 2026 — ALL RIGHTS RESERVED.